

Python & Bash Basics Assignment

Name: Kanaparthi Sai Dheeraj

Course: TuteDude DevOps Course

Module: Scripting (Python and Bash)

Objective

The objective of this assignment is to understand the basic concepts of Python programming, including conditional statements, dictionaries, and file handling. The assignment also provides hands-on practice in writing simple Python programs and understanding their execution.

Program 1: Grade Checker

Objective

To determine the student's grade based on the marks entered by the user using conditional statements.

Concepts Used

- Variables
- User Input
- Data Type Conversion
- if, elif, else

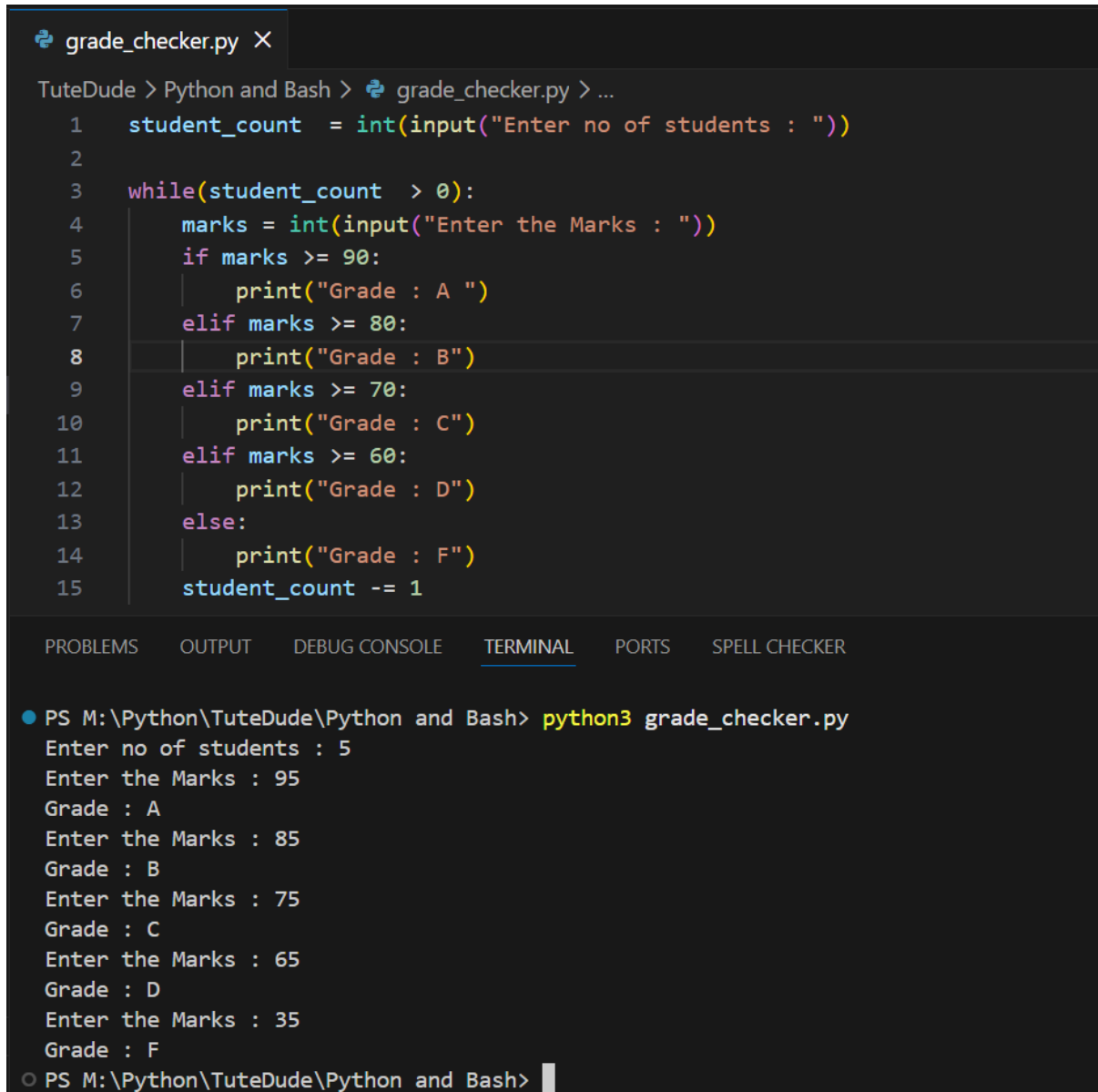
Explanation

The program accepts marks from the user and checks them using conditional statements. Depending on the entered marks, it displays the corresponding grade.

Grade Criteria

- 90 and above – Grade A
- 80 to 89 – Grade B
- 70 to 79 – Grade C
- 60 to 69 – Grade D
- Below 60 – Grade F

Output



The screenshot shows a code editor with a file named `grade_checker.py`. The code is a Python script that takes the number of students as input and then loops to get marks for each student, printing the corresponding grade. The terminal output shows the execution of the script with 5 students and marks of 95, 85, 75, 65, and 35, resulting in grades A, B, C, D, and F respectively.

```
grade_checker.py X
TuteDude > Python and Bash > grade_checker.py > ...
1  student_count = int(input("Enter no of students : "))
2
3  while(student_count > 0):
4      marks = int(input("Enter the Marks : "))
5      if marks >= 90:
6          print("Grade : A ")
7      elif marks >= 80:
8          print("Grade : B")
9      elif marks >= 70:
10         print("Grade : C")
11     elif marks >= 60:
12         print("Grade : D")
13     else:
14         print("Grade : F")
15     student_count -= 1

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER

● PS M:\Python\TuteDude\Python and Bash> python3 grade_checker.py
Enter no of students : 5
Enter the Marks : 95
Grade : A
Enter the Marks : 85
Grade : B
Enter the Marks : 75
Grade : C
Enter the Marks : 65
Grade : D
Enter the Marks : 35
Grade : F
○ PS M:\Python\TuteDude\Python and Bash>
```

Program 2: Student Grades Dictionary

Objective

To store, update, and display student grades using a Python dictionary.

Concepts Used

- Dictionary
- for Loop
- if-else
- Dictionary Operations

Explanation

The program allows the user to enter student names and grades, stores them in a dictionary, provides an option to update an existing student's grade, and finally displays all student records.

Output

```
student_Guide_Dict.py X
TuteDude > Python and Bash > student_Guide_Dict.py > ...
1  Student_grades = {}
2
3  n = int(input("Enter the number of students : "))
4
5  for i in range(n):
6      name = input("Enter the student name : ")
7      grade = input("Enter " + name + " grade : ")
8      Student_grades[name] = grade
9  update = input("Do you want to update a grade ? (yes/no) : ")
10
11 if update.lower() == "yes" :
12     name = input("Enter the name to update : ")
13     if name in Student_grades :
14         new_grade = input("Enter " + name + " new grade : ")
15         Student_grades[name] = new_grade
16     else:
17         print("Stduent Not Found! ")
18
19 print("\n Student Grades : ")
20
21 for name,grade in Student_grades.items():
22     print(name, " : ", grade)
23
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
● PS M:\Python\TuteDude\Python and Bash> python3 .\student_Guide_Dict.py
Enter the number of students : 3
Enter the student name : sai
Enter sai grade : A
Enter the student name : dheeraj
Enter dheeraj grade : B
Enter the student name : chary
Enter chary grade : C
Do you want to update a grade ? (yes/no) : yes
Enter the name to update : chary
Enter chary new grade :A

Student Grades :
sai : A
dheeraj : B
chary : A
○ PS M:\Python\TuteDude\Python and Bash>
```

Program 3: Write to a File

Objective

To create a text file and write data into it using Python.

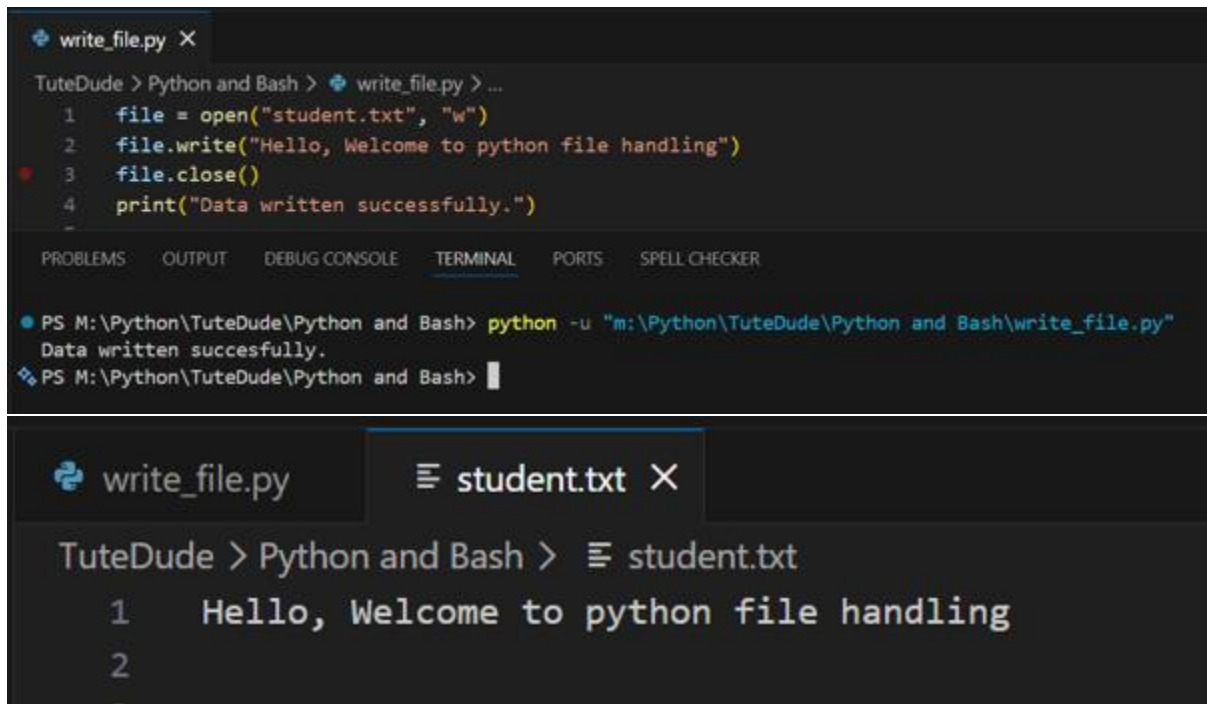
Concepts Used

- File Handling
- open()
- write()
- close()

Explanation

The program creates a file named student.txt (if it does not already exist) and writes text into the file using write mode ("w"). It then closes the file after writing.

Output



The screenshot displays a code editor with two panels. The top panel shows the source code for `write_file.py`:

```
1 file = open("student.txt", "w")
2 file.write("Hello, Welcome to python file handling")
3 file.close()
4 print("Data written successfully.")
```

Below the code, the terminal output shows the command `python -u "m:\Python\TuteDude\Python and Bash\write_file.py"` being executed, resulting in the message `Data written succesfully.` (note the typo in the original image).

The bottom panel shows the contents of the `student.txt` file, which has been opened in the editor. The file contains the text:

```
1 Hello, Welcome to python file handling
2
```

Program 4: Read from a File

Objective

To read the contents of a text file and display them on the screen.

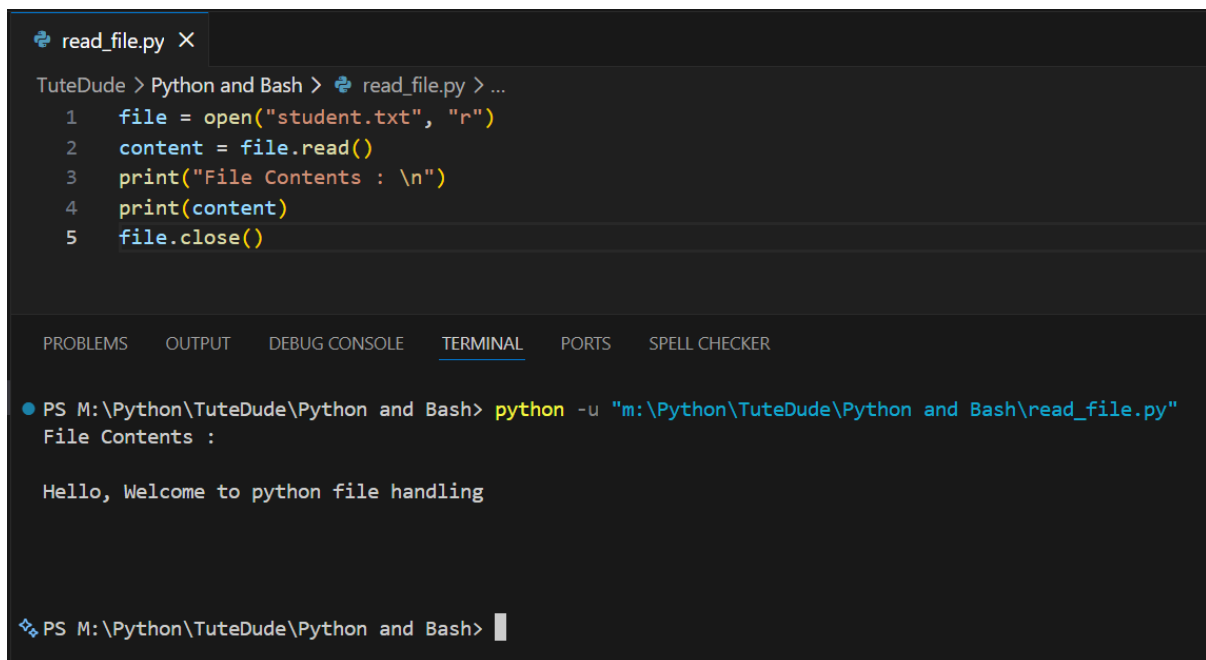
Concepts Used

- File Handling
- open()
- read()
- close()

Explanation

The program opens the student.txt file in read mode ("r"), reads its contents using the read() function, displays the contents on the screen, and closes the file.

Output



The screenshot shows a code editor with a file named `read_file.py`. The code in the editor is as follows:

```
1 file = open("student.txt", "r")
2 content = file.read()
3 print("File Contents : \n")
4 print(content)
5 file.close()
```

Below the code editor, the **TERMINAL** tab is active, showing the command to run the script and its output:

```
PS M:\Python\TuteDude\Python and Bash> python -u "m:\Python\TuteDude\Python and Bash\read_file.py"
File Contents :

Hello, Welcome to python file handling
```

The terminal prompt is currently `PS M:\Python\TuteDude\Python and Bash>`.

Conclusion

In this assignment, I learned the fundamentals of Python programming, including conditional statements, dictionaries, and file handling. I gained practical experience in creating Python programs, storing and updating data using dictionaries, writing data to files, and reading data from files. These concepts provide a strong foundation for further Python programming and DevOps automation.